

Section A:-

1. Explain how relational databases help maintain accuracy in expense records.

➔ Reduces Data Duplication: User and category information is stored only once in separate tables, avoiding repeated and inconsistent data.

Maintains Data Integrity: Primary keys and foreign keys ensure that every expense is linked to a valid user and category, preventing invalid records.

Ensures Data Consistency: Changes made to user or category information are reflected wherever they are referenced, keeping the data accurate.

Supports Accurate Queries: Relationships between tables allow users to retrieve correct expense details using SQL joins.

2. Why are constraints important in personal finance data?

➔ Constraints are important in personal finance data because they ensure the data is accurate, consistent, and reliable.

1. Maintain Data Integrity: Constraints prevent invalid or incorrect data from being entered into the database.

2. Prevent Duplicate Records: The UNIQUE constraint ensures that values such as email addresses are not duplicated.

3. Ensure Required Data: The NOT NULL constraint makes sure important fields like amount, expense_date, and category_id are never left empty.

4. Maintain Relationships: PRIMARY KEY and FOREIGN KEY constraints ensure that every expense is linked to a valid user and category.

5. Validate Data: CHECK constraints can prevent invalid values, such as negative expense amounts.
6. Improve Data Accuracy: Constraints help keep financial records correct and consistent, making reports and analysis more reliable.

3. How does GROUP BY help analyze spending patterns?

➔ SELECT category_name, SUM(amount) AS total_expense
FROM expenses INNER JOIN categories
ON expenses.category_id = categories.category_id
GROUP BY category_name;

4.Explain a scenario where rollback is required during expense entry.

➔ START TRANSACTION;
INSERT INTO expenses (user_id, category_id, amount, expense_date)
VALUES (1, 2, 5000, '2026-06-29'); -- An error occurs here
ROLLBACK;
Suppose a user is entering a new expense of ₹5,000 under the Rent category. While saving the record, the system detects that the category_id is invalid or the database connection fails.

5.How do views help users track monthly expenses efficiently?

➔ . Simplifies Data Access: Users can view monthly expense reports without writing complex SQL queries.
Shows Monthly Spending: A view can display the total expenses for each month, making it easy to monitor spending. Improves Performance: Frequently used reports can be accessed quickly through a view.

Enhances Security: Users can access only the required data without direct access to the underlying tables.

Supports Better Financial Planning: Monthly summaries help users identify spending trends and manage their budgets effectively.
CREATE VIEW monthly_expenses AS

```
SELECT  
MONTH(expense_date) AS month,  
SUM(amount) AS total_expense  
FROM expenses  
GROUP BY MONTH(expense_date);
```

6. Why use triggers for automatic category or balance updates?

```
➔ CREATE TRIGGER update_balance  
AFTER INSERT ON expenses  
FOR EACH ROW  
UPDATE users  
SET balance = balance - NEW.amount  
WHERE user_id = NEW.user_id;
```

Automatic Updates: Triggers can automatically update the account balance or category totals whenever a new expense is added, updated, or deleted.

Maintains Data Consistency: They ensure that related tables remain synchronized without requiring manual updates. Reduces Human Errors: Since updates happen automatically, the chances of forgetting to update balances or totals are minimized.

Saves Time: Users and developers do not need to write extra SQL queries to update related data.

Improves Data Accuracy: Triggers ensure that financial records are always up to date after every transaction.

Section B:-

Q1. DDL Understanding

- > 1.preventing “ orphaned ”data
- 2.ensuring data consistency(cascading action).

- 3.reliable data joins.
- 4.moving the logic to the database level.

If foreign keys are removed from the database schema, the system loses its ability to enforce data integrity automatically. The primary risk that emerges is the creation of Orphaned Records.

The Risk of Orphaned Records

When the foreign key constraint on `user_id` in the `expenses` table is removed, the database no longer verifies if an expense belongs to a valid user.

If a user account is deleted from the `users` table, any expenses previously associated with that `user_id` will remain entirely untouched in the `expenses` table.

This leads to several critical issues:

- ➔ Inaccurate Financial Reporting: Standard queries using `INNER JOIN` to calculate totals

or display user expenses will completely omit these records because the corresponding `user_id` no longer exists in the `users` table. This results in missing

financial data and distorted overall spending metrics.

- ➔ Wasted Storage & Data Pollution: The database accumulates "ghost" data—records

that consume space but have no context, meaning, or owner.

- ➔ Application Errors: If the application attempts to fetch or display details for that

specific expense, it will likely crash or throw errors when it tries to look up a user name or email that is missing from the system.

Q2.DML Operations:-

- Insert 5 users into the users table with unique names and emails.

				user_id	name	email	created_at
☐		Edit		Copy		Delete	1 Aarav Patel aarav.patel@example.com 2026-07-01
☐		Edit		Copy		Delete	2 Diya Shah diya.shah@example.com 2026-07-01
☐		Edit		Copy		Delete	3 Rohan Mehta rohan.mehta@example.com 2026-07-02
☐		Edit		Copy		Delete	4 Priya Joshi priya.joshi@example.com 2026-07-03
☐		Edit		Copy		Delete	5 Karan Desai Athrav.shah@example.com 2026-07-04

- Insert 3 categories (e.g., 'Food', 'Rent', 'Entertainment').

```
1 INSERT INTO categories (category_id, category_name)
2 VALUES
3 (1, 'Food'),
4 (2, 'Rent'),
5 (3, 'Entertainment');
```

←T→ ▼					category_id	category_name	
☐		Edit		Copy		Delete	1 Food
☐		Edit		Copy		Delete	2 Rent
☐		Edit		Copy		Delete	3 Entertainment

- Insert 10 expense records distributed across different users and categories.

		expense_id	user_id	category_id	amount	expense_date
☐	 Edit  Copy  Delete	1	1	1	250.00	2026-07-01
☐	 Edit  Copy  Delete	2	1	2	8000.00	2026-07-02
☐	 Edit  Copy  Delete	3	2	1	180.50	2026-07-02
☐	 Edit  Copy  Delete	4	2	3	450.00	2026-07-03
☐	 Edit  Copy  Delete	5	3	2	9500.00	2026-07-03
☐	 Edit  Copy  Delete	6	3	1	320.75	2026-07-04
☐	 Edit  Copy  Delete	7	4	3	600.00	2026-07-04
☐	 Edit  Copy  Delete	8	4	1	150.25	2026-07-05
☐	 Edit  Copy  Delete	9	5	2	7000.00	2026-07-05
☐	 Edit  Copy  Delete	10	5	3	900.00	2026-07-06

- Update one incorrect expense: Use the UPDATE command to change the amount of a specific expense_id.

```

1 UPDATE expenses
2 SET amount = 500.00
3 WHERE expense_id = 4;

```

		expense_id	user_id	category_id	amount	expense_date
☐	 Edit  Copy  Delete	1	1	1	250.00	2026-07-01
☐	 Edit  Copy  Delete	2	1	2	8000.00	2026-07-02
☐	 Edit  Copy  Delete	3	2	1	180.50	2026-07-02
☐	 Edit  Copy  Delete	4	2	3	500.00	2026-07-03
☐	 Edit  Copy  Delete	5	3	2	9500.00	2026-07-03
☐	 Edit  Copy  Delete	6	3	1	320.75	2026-07-04
☐	 Edit  Copy  Delete	7	4	3	600.00	2026-07-04
☐	 Edit  Copy  Delete	8	4	1	150.25	2026-07-05
☐	 Edit  Copy  Delete	9	5	2	7000.00	2026-07-05
☐	 Edit  Copy  Delete	10	5	3	900.00	2026-07-06

- Delete one expense: Remove a record where the amount is less than a specific value.

```
1 DELETE FROM expenses
2 WHERE amount < 200.00;
```

←T→			expense_id	user_id	category_id	amount	expense_date	
☐	 Edit	 Copy	 Delete	1	1	1	250.00	2026-07-01
☐	 Edit	 Copy	 Delete	2	1	2	8000.00	2026-07-02
☐	 Edit	 Copy	 Delete	4	2	3	500.00	2026-07-03
☐	 Edit	 Copy	 Delete	5	3	2	9500.00	2026-07-03
☐	 Edit	 Copy	 Delete	6	3	1	320.75	2026-07-04
☐	 Edit	 Copy	 Delete	7	4	3	600.00	2026-07-04
☐	 Edit	 Copy	 Delete	9	5	2	7000.00	2026-07-05
☐	 Edit	 Copy	 Delete	10	5	3	900.00	2026-07-06

3.Data Retrievel

- Display all expenses with details: Show the expense_date, amount, name (from users), and category_name (from categories) using INNER JOIN.

```
1 SELECT
2     e.expense_date,
3     e.amount,
4     u.name,
5     c.category_name
6 FROM expenses e
7 INNER JOIN users u
8 ON e.user_id = u.user_id
9 INNER JOIN categories c
10 ON e.category_id = c.category_id;
```

expense_date	amount	name	category_name
2026-07-01	250.00	Aarav Patel	Food
2026-07-02	8000.00	Aarav Patel	Rent
2026-07-03	500.00	Diya Shah	Entertainment
2026-07-03	9500.00	Rohan Mehta	Rent
2026-07-04	320.75	Rohan Mehta	Food
2026-07-04	600.00	Priya Joshi	Entertainment
2026-07-05	7000.00	Karan Desai	Rent
2026-07-06	900.00	Karan Desai	Entertainment

- Show total expense amount per category: Use SUM(amount) and GROUP BY category_name.

```
1 SELECT
2     c.category_name,
3     SUM(e.amount) AS total_expense
4 FROM expenses e
5 INNER JOIN categories c
6 ON e.category_id = c.category_id
7 GROUP BY c.category_name;
```

category_name	total_expense
Entertainment	2000.00
Food	570.75
Rent	24500.00

- Display users sorted by total spending: List user names and their total sum of expenses, ordered from highest to lowest.

```
1 SELECT
2     u.name,
3     SUM(e.amount) AS total_spending
4 FROM users u
5 INNER JOIN expenses e
6 ON u.user_id = e.user_id
7 GROUP BY u.name
8 ORDER BY total_spending DESC;
```

name	total_spending ▾ 1
Rohan Mehta	9820.75
Aarav Patel	8250.00
Karan Desai	7900.00
Priya Joshi	600.00
Diya Shah	500.00

Q4.Views

- Create a view named ActiveUsersView: This view should list the name and email of any user who has logged more than 5 individual expense records.

```
1 CREATE VIEW ActiveUsersView AS
2 SELECT
3     u.name,
4     u.email
5 FROM users u
6 INNER JOIN expenses e
7 ON u.user_id = e.user_id
8 GROUP BY u.user_id, u.name, u.email
9 HAVING COUNT(e.expense_id) > 5;
```

- Query the view: Write a simple SELECT statement to show all data from ActiveUsersView.

```
1 SELECT * FROM ActiveUsersView;
```

Section c :- mini project

Crud program

1.Insert data

```
1 INSERT INTO users (name, email)
2 VALUES ('Vijay', 'vijay@gmail.com');
```

2.update

```
1 UPDATE nimesh
2 set "vijay"
3 WHERE expensesid="1";
```

3.Delete

```
1 DELETE from nimesh "vijay"  
2 WHERE expensesid="1";
```

3. Write a store procedure to calculate monthly user expense.

user_id	month	year	total_monthly_expense
2	6	2026	8350.00

4. Commit and rollback

←T→	expense_id	user_id	category_id	amount	expense_date
☐ Edit ☐ Copy ☐ Delete	11	1	2	500.00	2026-06-11